API仕様書

テスト画面アクセス

項目	値
URL	https://image-analysis.example-cloud.com
アクセス条件	許可IP (203.0.113.1) からのみ
認証	Cognito ユーザーアカウント (メール/パスワード)
対応ブラウザ	Chrome / Edge / Firefox (最新版)

Cognito認証の取得手順

テスト画面から (自動)

1. https://image-analysis.example-cloud.com にアクセス
2. Amplify Authenticatorのログイン画面が表示
3. メールアドレス・パスワードを入力してサインイン
4. 以降のAPI呼び出しは自動的にJWTが付与される

API直接呼び出し用 (手動トークン取得)

```
# Cognito トークン取得
aws cognito-idp initiate-auth \
  --client-id <USER_POOL_CLIENT_ID> \
  --auth-flow USER_PASSWORD_AUTH \
  --auth-parameters USERNAME=<email>,PASSWORD=<password> \
  --region ap-northeast-1

# レスポンスから IdToken を取得して使用
export COGNITO_TOKEN="eyJraWQi..."
```

APIキー取得手順 (ワークフロー連携用)

1. AWSコンソール → API Gateway → APIキー → 対象キーの値を取得
2. または AWS CLI で取得:

```
aws apigateway get-api-keys --include-values \
  --region ap-northeast-1 \
  --query 'items[?contains(name, `api-key`)].{name: name, value: value}' \
  --output table
```

各エンドポイント呼び出しサンプル

POST /api/analyze

curl (テスト画面用: Cognito JWT)

```
# 画像をbase64エンコード
IMAGE_B64=$(base64 -w 0 photo.jpg)

curl -X POST https://image-analysis.example-cloud.com/api/analyze \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $COGNITO_TOKEN" \
  -d "{
    \"image\": \"$IMAGE_B64\",
    \"target\": \"crack\",
    \"model\": \"gpt-4.1-mini\",
    \"detail\": \"low\"
  }"
```

curl (ワークフロー用: APIキー + Base64)

```
curl -X POST https://<api-gateway-domain>/prod/workflow/analyze \
  -H "Content-Type: application/json" \
  -H "x-api-key: $API_KEY" \
  -d "{
    \"data\": \"$(base64 -w 0 photo.jpg)\",
    \"filename\": \"photo.jpg\",
    \"target\": \"crack\"
  }"
```

curl (ワークフロー用: APIキー + S3キー参照)

```
curl -X POST https://<api-gateway-domain>/prod/workflow/analyze \
  -H "Content-Type: application/json" \
  -H "x-api-key: $API_KEY" \
  -d '{
    "image_bucket": "<bucket-name>",
    "image_key": "tmp/message-id/photo.jpg",
    "target": "crack"
  }'
```

Python

```
import base64
import requests
```

```
with open("photo.jpg", "rb") as f:
    image_b64 = base64.b64encode(f.read()).decode()

response = requests.post(
    "https://image-analysis.example-cloud.com/api/analyze",
    headers={
        "Content-Type": "application/json",
        "Authorization": f"Bearer {cognito_token}"
    },
    json={
        "image": f"{image_b64}",
        "target": "crack",
        "detail": "low"
    }
)
print(response.json())
```

JavaScript (fetch)

```
const file = document.querySelector('input[type="file"]').files[0]
const reader = new FileReader()
reader.onload = async () => {
  const response = await fetch('/api/analyze', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${idToken}`
    },
    body: JSON.stringify({
      image: reader.result,
      target: 'crack',
      detail: 'low'
    })
  })
  const result = await response.json()
  console.log(result)
}
reader.readAsDataURL(file)
```

GET /api/health

```
curl https://image-analysis.example-cloud.com/api/health
# => {"status": "ok", "timestamp": "2026-03-04T12:00:00Z"}
```

エラーコード一覧と対処法

エラーレスポンスは `{"error": "メッセージ"}` 形式で返却される。

HTTP	コード	原因	対処法
400	MISSING_IMAGE	image/dataが未指定	いずれかを指定する
400	MISSING_TARGET	targetが未指定	targetを指定する
400	INVALID_IMAGE	base64デコード失敗	画像を再エンコード
400	IMAGE_TOO_LARGE	20MB超過	画像を圧縮する
401	UNAUTHORIZED	JWT無効/期限切れ	再ログインしてトークン再取得
403	FORBIDDEN	APIキー無効またはIP制限	キーとIPを確認
500	AI_ERROR	OpenAI API障害	時間を置いてリトライ
504	TIMEOUT	処理タイムアウト	detail=lowに変更、画像縮小

ワークフロー連携 (external-system)

ワークフローJSON定義サンプル

external-systemの `UploadByHttp` タスクを使用して本APIを呼び出す。

```
{
  "name": "画像解析ワークフロー",
  "subscribe": {
    "topic": "NotificationTopic",
    "filter": {
      "subject": { "contains": "inspection" }
    }
  },
  "tasks": [
    {
      "type": "GetMessage",
      "name": "メッセージ取得"
    },
    {
      "type": "ChoiceAttachments",
      "name": "添付画像選択",
      "config": {
        "mimeTypes": ["image/jpeg", "image/png"]
      }
    },
    {
      "type": "UploadByHttp",
      "name": "画像解析API呼び出し",
      "config": {
        "url": "https://<api-gateway-domain>/prod/workflow/analyze",
        "method": "POST",
        "asJson": true,
        "headers": {
          "x-api-key": "<API_KEY>"
        },
```

```
          "bodyTemplate": {
            "target": "crack",
            "model": "gpt-4.1-mini",
            "detail": "low"
          }
        }
      }
    ]
  }
```

UploadByHttpの動作

1. `ChoiceAttachments` で選択された添付ファイル (`attachments[0]`) を取得
2. `base64.b64encode(attachment.payload)` でBase64化
3. `bodyTemplate` の内容に `data` (Base64) と `filename` を自動追加
4. 指定URLにPOST送信